

Building an 8-bit Serial Data Storage System

Goal: Build reusable 8-bit systems that will later form part of an 8-bit CPU

Part 1 — 8-bit Serial-In Parallel-Out (SIPO)

Objective: Convert a serial bit stream into an **8-bit parallel output** and display it.

Step 1.1 — Create a new circuit

1. Create a new Logisim circuit.
2. Rename it **SIPO_8bit**.
3. Add:
 - a. One **clock**
 - b. One **serial input bit**
 - c. Eight **D flip-flops**

Step 1.2 — Build the shift register

1. Make the **SIPO** circuit given in **Lecture 9** slides.
2. Label the inputs (**clk** for clock, **D** for data) and the outputs **Q7 ... Q0** (LSB=Q0).

Step 1.3 — Add a parallel output display

1. Combine the eight outputs into an **8-bit bus** (splitter).
2. Connect the bus to an **8-bit output** (label it Output).

Step 1.4 — Test

1. Run the clock manually.
2. Enter a known bit pattern serially.
3. Confirm the parallel output updates **one bit per clock**.

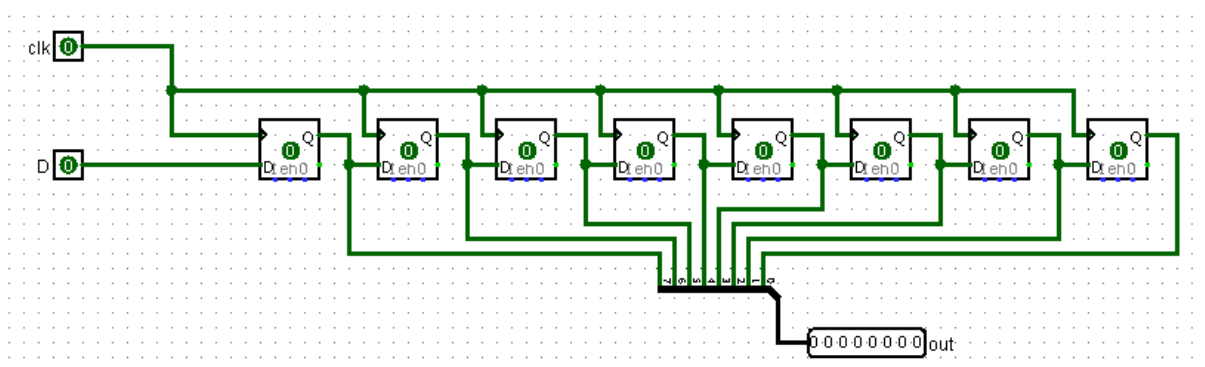


Figure 1. SIPO Circuit.

Step 1.5 — Circuit Layout

1. Amend the circuit layout with appropriate labelling of the inputs and outputs.

Step 1.6 — Integration

1. Place the SIPO circuit (sub-system) in the main circuit.
2. Add and connect:
 - a. One **clock**, One **serial input**, One **8-bit binary display**.
3. Test the system.

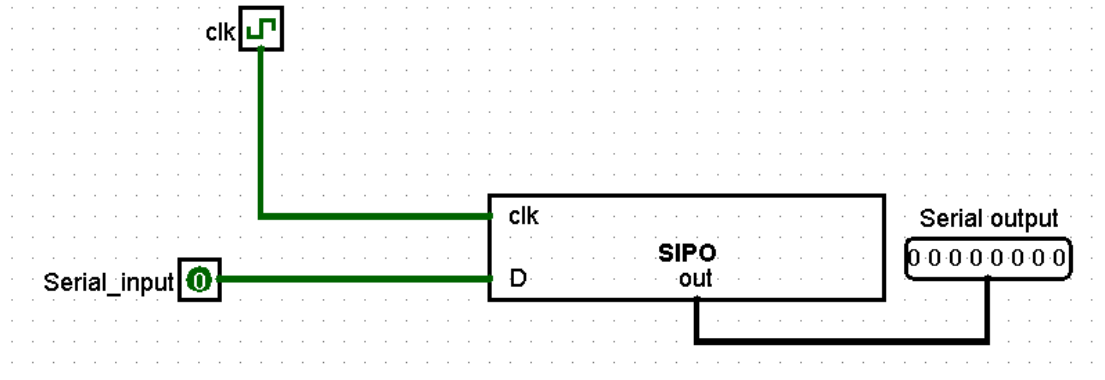


Figure 2. The main circuit with the SIPO and a display.

Part 2 — 8-bit Parallel-In Parallel-Out (PIPO) Register

Objective: Build an **8-bit register** and observe.

Step 2.1 — Create a new circuit

1. Create a new circuit named **PIPO_8bit**.
2. Add:
 - a. Eight D flip-flops
 - b. One clock
 - c. One 8-bit input
 - d. One 8-bit output

Step 2.2 — Build the register

1. Make the **PIPO** circuit given in **Lecture 9** slides.
2. Combine the eight outputs into an **8-bit bus** (splitter).
3. Connect the bus to an **8-bit output** (label it Output).
4. Label appropriately.

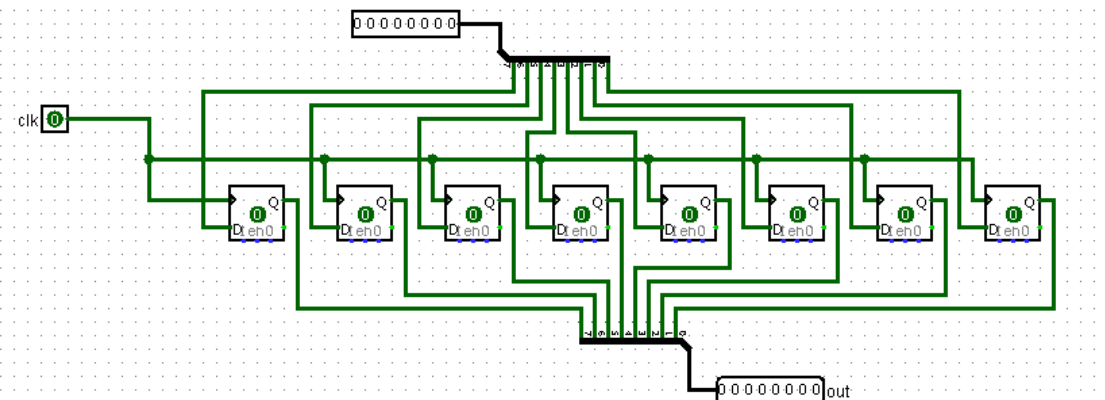


Figure 3. PIPO Circuit.

Step 2.3 — Integration and Test behaviour

1. Amend the Circuit Appearance appropriately.
2. Place the SIPO circuit (sub-system) in the main circuit.
3. Connect the **output of SIPO_8bit** directly to the PIPO input.
4. Connect the clock signal.
5. Add and connect an **8-bit binary display** to the output of the PIPO.
6. Run the clock.

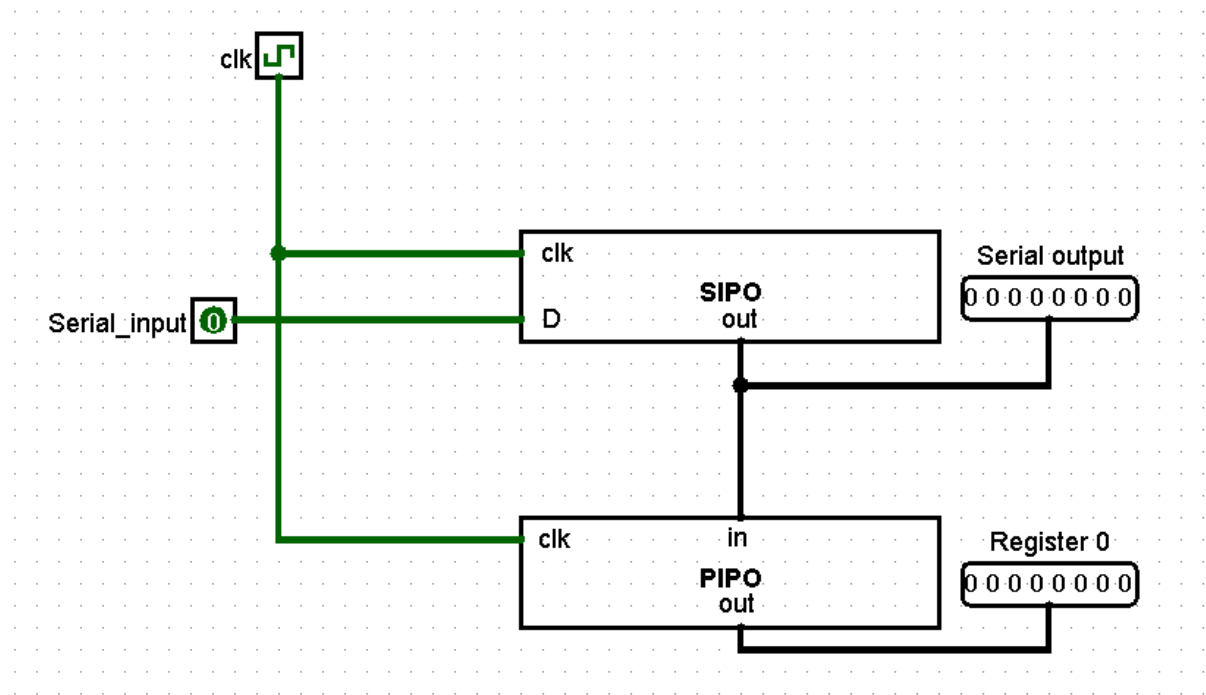


Figure 4. The main circuit with the SIPO, PIPO registers and displays.

Observation

- What does the PIPO display show at every clock cycle?
- Does it store a complete 8-bit word (the last pack)?

Why Control Is Needed?

- A register must **not** load data on every clock cycle.
- To store *complete* 8-bit words, the PIPO must load **only after 8 bits have arrived**.
- What kind of a device do we need?

Part 4 — 3-bit Counter for Word Timing

Objective: Create a counter that counts **8 clock cycles**, then signals that a full word is ready.

Step 4.1 — Create a new circuit

1. Create a new circuit named **CountUp_3bit**.

Step 4.2 — Build the counter

1. Implement a **3-bit binary up-counter** (counts 0 → 7).
2. When the count reaches 7, generate a signal called **word_valid**.

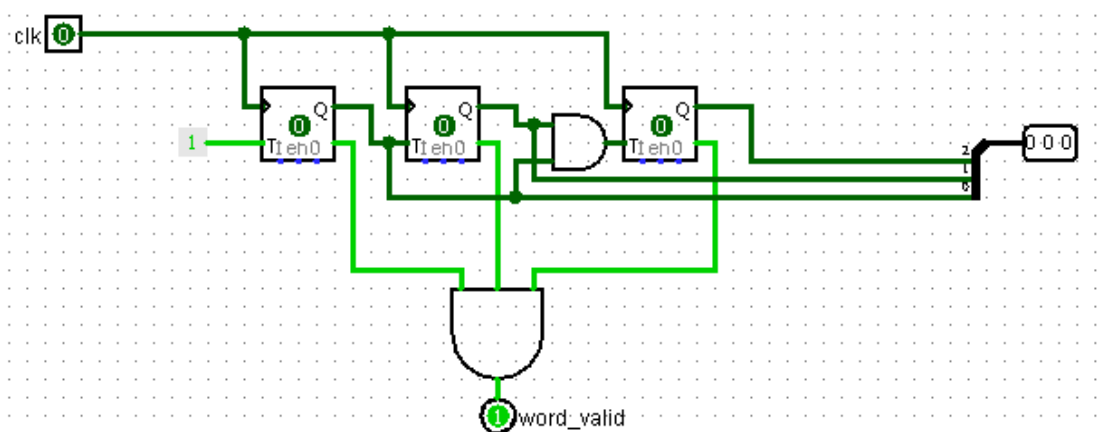


Figure 5. CountUp_3bit Circuit. Pulse gives a “1” every 8 clock cycles.

Part 5 — Controlled Loading of the 8-bit Register

Objective: Load the PIPO only when a full 8-bit word is ready.

Step 5.1 — Modify the Main Circuit

1. Add the **CountUp_3bit** in the Main, and connect its pulse generator to the **clk** input of the PIPO_8bit.

Step 5.3 — Test again

1. Run the clock continuously.
2. Observe the PIPO display.

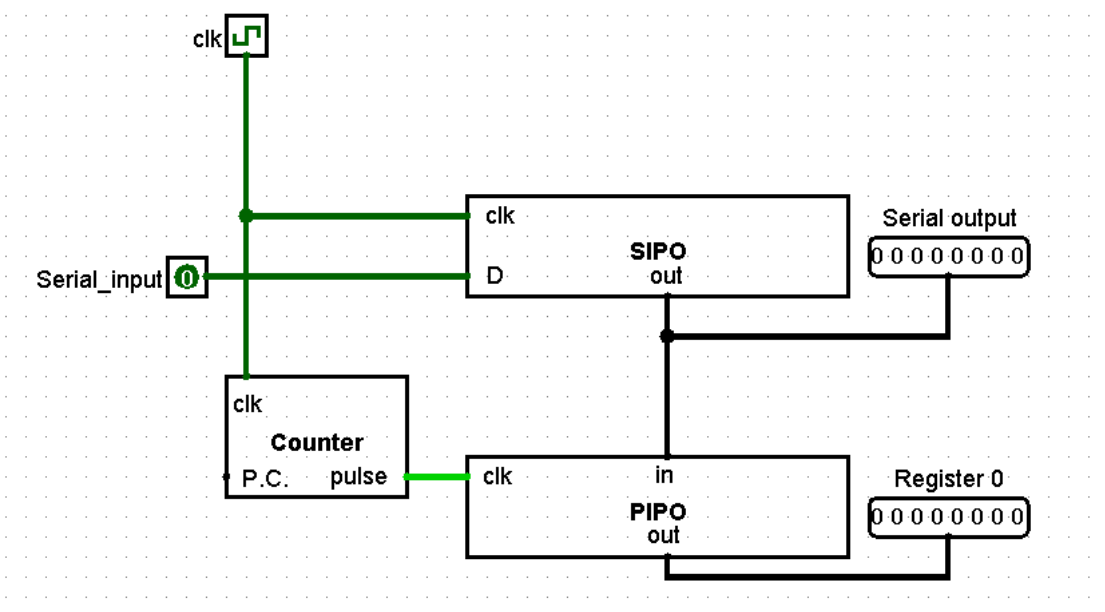


Figure 6. The main circuit with a 3bit counter pulse generator.

Part 6 — Storing the Last Three 8-bit Words

Objective: Extend the system to keep the **last three 8-bit packs**.

Step 6.1 — Add PIPO in sequence

1. Add three instances of the PIPO_8bit circuit in sequence:
 - a. New 8-bit word → Register 0
 - b. Register 0 → Register 1
 - c. Register 1 → Register 2

Step 6.3 — Add displays

1. Attach an 8-bit display to each register output.
2. Label them accordingly.

Step 6.4 — Final test

1. Input multiple 8-bit words.
2. Confirm that only the **last three complete words** are stored.

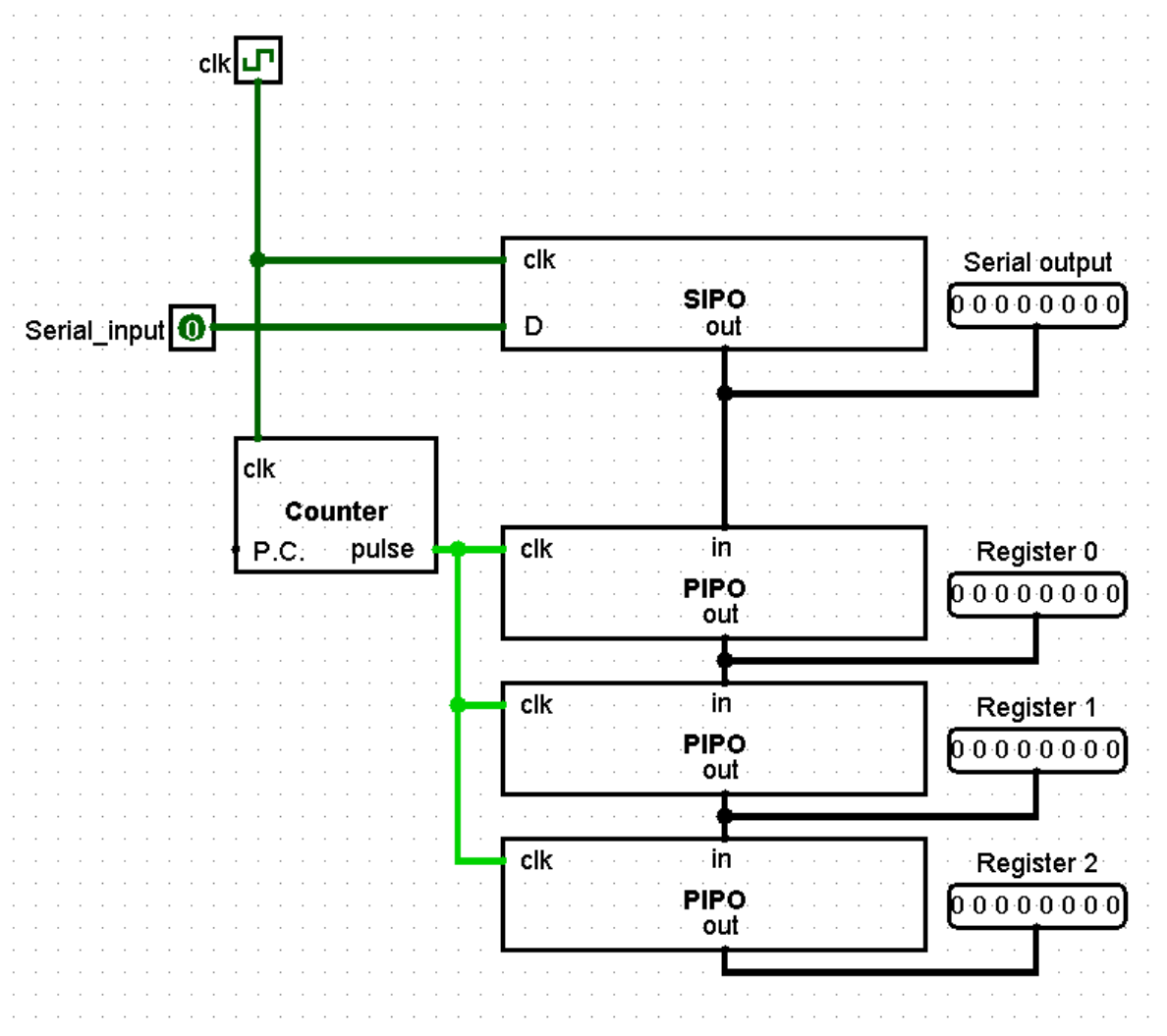


Figure 7. The full circuit in main.